Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase V · Alineamiento, aprendizaje por refuerzo, agentes e interpretabilidad

Módulo 22

Agentes, herramientas y RAG

Aprendizaje en contexto, recuperación de información y agentes con herramientas

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 22: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
θ, p_θ	parámetros del modelo de lenguaje y distribución que define
C, $(x_i, y_i)_{i=1}^{n}$	contexto (*prompt*) y demostraciones que contiene
$\mathcal{C}$, $N = \lvert\mathcal{C}\rvert$	colección de documentos o trozos y su tamaño
q, d, t	consulta, documento y término
$f_{t,d}$, df_t, idf_t	frecuencia de t en d, número de documentos que contienen t y frecuencia inversa
$\lvert d\rvert$, $\bar{\ell}$, $\ell = \lvert d\rvert/\bar{\ell}$	longitud de d, longitud media y longitud relativa
k_1, b	parámetros de saturación y de normalización por longitud de BM25
$\mathcal{R}$, T_k	conjunto de documentos relevantes y k primeros documentos del ranking
$\mathrm{P@}\,k$, $\mathrm{R@}\,k$	precisión y exhaustividad en el corte k
E_Q, E_P	codificadores de consultas y de pasajes
z, $p_\eta(z \mid x)$, $p_\theta(y \mid x, z)$	documento recuperado, recuperador y generador de RAG
$\mathcal{S}$, $\mathcal{A}$, Ω	estados, acciones y observaciones de un agente
h_t, b_t, π	historia, estado de creencia y política
p, n	probabilidad de éxito por paso y número de pasos
$\ln$, $\log_{10}$	logaritmo natural y decimal

Índice

Sobre este resumen **ii**

1 Aprendizaje en contexto y razonamiento **1**
1.1 Adaptación mediante el contexto . . . 1
1.2 Cabezas de inducción . . . 2
1.3 La atención como aprendizaje sobre las demostraciones . . . 2
1.3.1 Regresión por núcleos . . . 2
1.3.2 Un paso de descenso de gradiente . . . 3
1.4 Cadena de pensamiento y autoconsistencia . . . 3
1.5 El coste del contexto . . . 4

2 Recuperación léxica: tf-idf, ordenación probabilística y BM25 **6**
2.1 El problema de la recuperación . . . 6
2.2 Frecuencia de término y frecuencia inversa de documento . . . 6
2.3 El modelo de espacio vectorial . . . 7
2.4 El principio de ordenación por probabilidad . . . 7
2.5 El modelo de independencia binaria . . . 7
2.6 BM25 . . . 8

3 Evaluación de la recuperación y de los modelos **9**
3.1 Conjuntos de resultados . . . 9
3.2 Rankings . . . 9
3.3 Evaluación de modelos de lenguaje . . . 10
3.4 Contaminación, saturación y jueces . . . 11

4 Recuperación densa y búsqueda de vecinos más próximos **12**
4.1 Codificadores duales . . . 12
4.2 Coseno, producto escalar y distancia euclídea . . . 13
4.3 Búsqueda exacta y aproximada . . . 13
4.3.1 Cuantización de producto . . . 13
4.3.2 Grafos de mundo pequeño navegables y jerárquicos . . . 14
4.4 Búsqueda híbrida . . . 14

5 Generación aumentada por recuperación **15**
5.1 Memoria paramétrica y no paramétrica . . . 15
5.2 Los modelos RAG-Sequence y RAG-Token . . . 15
5.3 Troceo . . . 16
5.4 El *prompt* y su coste . . . 16
5.5 Evaluación de un sistema RAG . . . 17

6 Agentes y uso de herramientas **18**
6.1 Agentes racionales . . . 18
6.2 El agente como proceso de decisión parcialmente observable . . . 18
6.3 Llamadas a herramientas . . . 19
6.3.1 Toolformer: aprender cuándo llamar . . . 20
6.3.2 ReAct: razonar y actuar . . . 20
6.4 Fiabilidad de una cadena de pasos . . . 21
6.5 Riesgos: inyección de instrucciones . . . 21

Bibliografía **23**

Tema 1

Aprendizaje en contexto y razonamiento

Un modelo de lenguaje preentrenado puede resolver tareas nuevas a partir de unos pocos ejemplos escritos en su entrada, sin que se modifique ninguno de sus parámetros. Este tema define este aprendizaje en contexto, describe el circuito de atención que lo explica en parte, demuestra dos resultados que lo relacionan con algoritmos clásicos de aprendizaje (la regresión por núcleos y el descenso de gradiente) y estudia el razonamiento escrito paso a paso y su coste. Se siguen el artículo de GPT-3 de Brown *et al.* [5], el de las cabezas de inducción de Olsson *et al.* [26], el de von Oswald *et al.* [27], la §11.2 de Zhang *et al.* [41], los artículos de la cadena de pensamiento [36, 20] y de la autoconsistencia [35], y la clase 9 de CS224N [38].

1.1 Adaptación mediante el contexto

Definición 1.1 (Demostraciones y aprendizaje en contexto). Sea p_θ un modelo de lenguaje autorregresivo con parámetros fijos θ y sea una tarea definida por pares de entrada y salida (x, y). Un *prompt* con k *demostraciones* es la secuencia

$$C_k = \big(I,\ x_1,\ y_1,\ \ldots,\ x_k,\ y_k\big),$$

formada por una instrucción I, posiblemente vacía, y k pares de la tarea, concatenados con separadores fijos. Ante una entrada nueva x, el modelo responde con $\hat{y} = \arg\max_y p_\theta(y \mid C_k, x)$ o con una muestra de esa distribución. Se habla de *cero ejemplos* (*zero-shot*) si $k = 0$ y de *pocos ejemplos* (*few-shot*) si $k \geq 1$. Hay *aprendizaje en contexto* cuando la calidad de $\hat{y}$ mejora al aumentar k, con θ fijo.

La definición 1.1 es la de Brown *et al.* [5, §2], que contraponen tres regímenes (cuadro 1.1). En el aprendizaje en contexto, la información de la tarea entra solo por el condicionamiento: la distribución $p_\theta(\cdot \mid C_k, x)$ cambia porque cambia C_k, no porque cambie θ. Al borrar el *prompt*, la adaptación desaparece.

Régimen	Contenido del contexto	Parámetros
Cero ejemplos	Solo la instrucción I	Fijos
Pocos ejemplos	I y k demostraciones (de una a unas decenas)	Fijos
Ajuste fino (módulo 20)	Nada específico de la tarea	Se actualizan con miles de ejemplos

Cuadro 1.1: Tres formas de adaptar un modelo a una tarea [5, §2].

Observación 1.2. Brown *et al.* observan que la ganancia de pasar de cero a pocos ejemplos crece con el tamaño del modelo [5]. Una explicación intuitiva es que el preentrenamiento consiste

en predecir el token siguiente en textos que contienen listas, tablas y pares de pregunta y respuesta: continuar un patrón presente en el contexto reduce la pérdida, y el modelo aprende a hacerlo. Los dos apartados siguientes formalizan dos mecanismos concretos que lo permiten.

1.2 Cabezas de inducción

Olsson *et al.* [26] identifican en los Transformers un circuito de dos cabezas de atención que completa el patrón $[A][B]\dots[A] \to [B]$: si el token actual ya apareció antes, se predice el token que lo siguió entonces.

Definición 1.3 (Circuito de inducción idealizado). Sea $x_{1:t}$ una secuencia sobre un vocabulario $\mathcal{V}$ y sea $e(v)$, $v \in \mathcal{V}$, una base ortonormal de incrustaciones. El circuito consta de dos cabezas en capas consecutivas:

(i) una *cabeza de token previo*, que en cada posición $j \geq 2$ atiende a la posición $j-1$ y escribe $e(x_{j-1})$ en un subespacio propio del flujo residual, de modo que el estado de la posición j contiene $e(x_j)$ y $e(x_{j-1})$;

(ii) una *cabeza de inducción*, cuya consulta en la posición t es $\beta\, e(x_t)$, con $\beta > 0$, cuya clave en la posición j es $e(x_{j-1})$ y cuyo valor en j es $e(x_j)$.

Proposición 1.4 (Qué calcula el circuito). *Sea $J = \{\, j \in \{2,\dots,t\} : x_{j-1} = x_t \,\}$, el conjunto de posiciones que siguen a una aparición anterior del token actual. Si $J \neq \varnothing$, cuando $\beta \to \infty$ la salida de la cabeza de inducción en la posición t tiende a*

$$o_t = \frac{1}{|J|} \sum_{j \in J} e(x_j),$$

cuya coordenada en $e(v)$ es la frecuencia relativa con que v ha seguido a x_t en el contexto.

Si la proyección de salida lleva o_t a los logits del vocabulario, el token más probable es el sucesor más frecuente de x_t en el contexto. Es la regla de un modelo de bigramas (módulo 1) estimado *sobre el propio contexto*, no sobre el corpus de entrenamiento.

Observación 1.6. Olsson *et al.* observan que estas cabezas se forman de manera abrupta durante el entrenamiento, al mismo tiempo que mejora bruscamente la pérdida en los tokens alejados del principio del contexto, y proponen que versiones «borrosas» del mismo mecanismo (que copian patrones parecidos y no solo idénticos) explican una parte importante del aprendizaje en contexto [26]. Es una hipótesis apoyada por experimentos, no un teorema.

1.3 La atención como aprendizaje sobre las demostraciones

Las demostraciones (x_i, y_i) de un *prompt* son un pequeño conjunto de entrenamiento. Dos resultados muestran que una capa de atención puede aplicarles un algoritmo de aprendizaje clásico.

1.3.1 Regresión por núcleos

En el módulo 13 se vio que la agrupación por atención generaliza el estimador de Nadaraya–Watson, $\hat{f}(q) = \sum_i \kappa(q,k_i) v_i / \sum_l \kappa(q,k_l)$ [41, §11.2]. El caso de claves normalizadas es el que conecta con la atención de los Transformers.

Proposición 1.7 (Núcleo gaussiano sobre la esfera). *Sean $q, k_1, \dots, k_n \in \mathbb{R}^d$ con $\|q\| = \|k_i\| = 1$, valores v_i y el núcleo gaussiano de anchura $\sigma > 0$, $\kappa_\sigma(q,k) = \exp\big(-\|q-k\|^2/(2\sigma^2)\big)$. Entonces*

$$\hat{f}(q) = \sum_{i=1}^{n} \operatorname{softmax}\Big(\frac{q\cdot k_1}{\sigma^2}, \dots, \frac{q\cdot k_n}{\sigma^2}\Big)_i v_i,$$

es decir, el estimador es una atención de producto escalar con temperatura σ^2.

1.3.2 Un paso de descenso de gradiente

Teorema 1.9 (Atención lineal y descenso de gradiente [27]). *Sean demostraciones (x_i, y_i), $i = 1, \dots, n$, con $x_i \in \mathbb{R}^d$ e $y_i \in \mathbb{R}$, y la pérdida cuadrática $L(w) = \frac{1}{2}\sum_i (w^\top x_i - y_i)^2$. Un paso de descenso de gradiente desde $w_0 = 0$ con tasa η produce, para una consulta x_q, la predicción*

$$\hat{y}_q = w_1^\top x_q = \eta \sum_{i=1}^{n} y_i \, (x_i^\top x_q),$$

que coincide con la salida de una atención lineal (sin softmax) con consulta x_q, claves x_i, valores y_i y factor de escala η.

Observación 1.11. Von Oswald *et al.* construyen pesos de atención lineal que implementan varios pasos de este tipo y observan que Transformers entrenados sobre tareas de regresión lineal en contexto llegan a soluciones próximas a esa construcción [27]. El resultado ofrece una intuición sobre el mecanismo; no demuestra que los modelos de lenguaje hagan esto con texto, donde intervienen la softmax, las proyecciones aprendidas y muchas capas.

1.4 Cadena de pensamiento y autoconsistencia

Definición 1.12 (Cadena de pensamiento). Un *prompt de cadena de pensamiento* incluye demostraciones de la forma (x_i, r_i, y_i), donde r_i es un razonamiento intermedio escrito en lenguaje natural [36]. Ante x, el modelo genera primero un razonamiento $r \sim p_\theta(\cdot \mid C, x)$ y después la respuesta $y \sim p_\theta(\cdot \mid C, x, r)$. La distribución de la respuesta es la marginal

$$p_\theta(y \mid C, x) = \sum_{r} p_\theta(r \mid C, x)\, p_\theta(y \mid C, x, r).$$

Wei *et al.* mostraron que este tipo de *prompt* mejora mucho los problemas aritméticos y de razonamiento simbólico en modelos grandes [36], y Kojima *et al.* obtuvieron un efecto parecido sin demostraciones, con solo añadir la frase «pensemos paso a paso» (en inglés) [20].

Observación 1.13 (Cómputo por token). Un Transformer de L capas realiza una cantidad fija de operaciones por token generado, unas $2N$ para un modelo de N parámetros (módulo 14). Si un problema requiere más pasos de cálculo de los que caben en una pasada, escribirlos como tokens le concede más cómputo, y cada token escrito queda disponible como memoria de trabajo para los siguientes. Con $N = 7{\cdot}10^9$, 200 tokens de razonamiento cuestan $2{\cdot}7{\cdot}10^9{\cdot}200 = 2{,}8 \cdot 10^{12}$ operaciones adicionales. Los modelos de razonamiento del módulo 21 se entrenan con aprendizaje por refuerzo para generar razonamientos más largos y útiles.

Definición 1.14 (Autoconsistencia). Dado x, se muestrean m razonamientos independientes $r^{(1)}, \dots, r^{(m)}$ con temperatura positiva, se extrae la respuesta final $a^{(j)}$ de cada uno y se devuelve la más frecuente [35].

Proposición 1.15 (La autoconsistencia aproxima la moda de la marginal). *Si cada par $(r^{(j)}, a^{(j)})$ se muestrea de $p_\theta(r, y \mid C, x)$, la frecuencia relativa de cada respuesta y converge casi seguramente a $p_\theta(y \mid C, x)$ cuando $m \to \infty$. En consecuencia, si la moda de la marginal es única, la respuesta devuelta coincide con ella para m suficientemente grande.*

La respuesta elegida con un único razonamiento voraz maximiza $p_\theta(y \mid C, x, r)$ para un r concreto; la autoconsistencia, en cambio, apunta a la respuesta más probable una vez promediados todos los razonamientos. El caso binario admite una cota explícita, que usa la desigualdad de Hoeffding [13]: si $X_1, \ldots, X_m$ son independientes con valores en $[0, 1]$ y media μ, entonces $P(\bar{X} - \mu \le -\varepsilon) \le e^{-2m\varepsilon^2}$ para todo $\varepsilon > 0$.

Proposición 1.16 (Voto mayoritario). *Supóngase que cada razonamiento conduce a la respuesta correcta con probabilidad $p > 1/2$, de forma independiente, y que m es impar. La probabilidad de que la mayoría de las m respuestas sea incorrecta es*

$$\sum_{j=0}^{(m-1)/2} \binom{m}{j} p^j (1-p)^{m-j} \;\le\; e^{-2m(p-1/2)^2}.$$

Observación 1.18 (Árbol de pensamientos). Yao *et al.* generalizan la cadena a una búsqueda: el razonamiento se divide en pasos («pensamientos»), el modelo propone varias continuaciones de cada estado parcial y el propio modelo, con otro *prompt*, evalúa cada una; una búsqueda en anchura que conserva los b mejores estados por nivel, o una búsqueda en profundidad con retroceso, decide cuáles se expanden [40]. El coste es el de generar y evaluar todos los estados explorados.

1.5 El coste del contexto

El contexto no es gratuito. Tiene un coste en memoria, en cómputo y en calidad.

Proposición 1.19 (Memoria de la caché KV). *Un Transformer de n_{capas} capas que guarda claves y valores de dimensión total d_{kv} por capa, con β bytes por número, ocupa $2\, n_{\text{capas}}\, d_{kv}\, \beta$ bytes por token de contexto (módulo 19).*

Con la configuración pública de Llama 2 7B (32 capas, $d = 4096$, sin atención de consultas agrupadas) en 16 bits, son $2 \cdot 32 \cdot 4096 \cdot 2 = 524\,288$ bytes, es decir, 0,5 MiB por token: 2 GiB para 4096 tokens y 52,4 GB para 10^5, más que los 14 GB de los pesos. En cómputo, la matriz de puntuaciones de la atención tiene n^2 entradas (módulo 13): multiplicar el contexto por 10 multiplica por 100 esas puntuaciones y solo por 10 el coste de los bloques MLP.

Observación 1.20 (Ventanas largas y uso del contexto). Las ventanas de contexto públicas han crecido mucho: Anthropic documentó 200 000 tokens para la familia Claude 3 [1], y el informe técnico de Gemini 1.5 evalúa contextos de hasta 10 millones de tokens en experimentos [9]. Pero disponer de la información no garantiza usarla: Liu *et al.* observaron que varios modelos aprovechan peor la información situada en mitad de un contexto largo que la situada al principio o al final [22].

La consecuencia práctica motiva el resto del módulo. En lugar de pegar 8000 tokens de apuntes en el contexto, pueden recuperarse 3 fragmentos de 200 tokens y añadir 50 tokens de pregunta: 650 tokens, $8000/650 = 12{,}3$ veces menos, y unas $12{,}3^2 \approx 151$ veces menos

puntuaciones de atención. Elegir bien esos fragmentos es el problema de la recuperación de información (temas 2 a 4).

Tema 2

Recuperación léxica: tf-idf, ordenación probabilística y BM25

Un sistema RAG necesita, ante cada pregunta, los fragmentos de una colección que mejor la responden. Este es el problema clásico de la recuperación de información, que dispone de modelos con fundamento probabilístico y de décadas de evaluación. Este tema presenta los modelos léxicos, que comparan consulta y documento a través de los términos que comparten: el modelo de espacio vectorial con pesos tf-idf, el principio de ordenación por probabilidad, el modelo de independencia binaria y la función BM25. Se siguen los capítulos 6 y 11 de Manning, Raghavan y Schütze [24], la monografía de Robertson y Zaragoza [30] y la §11.1 de Jurafsky y Martin [17].

2.1 El problema de la recuperación

Definición 2.1 (Recuperación ordenada). Sea una *colección* $\mathcal{C} = \{d_1, \ldots, d_N\}$ de documentos (o trozos de documento) y una *consulta* q. Para cada par existe un juicio de *relevancia* $\mathrm{rel}(q, d) \in \{0, 1\}$, y $\mathcal{R}_q = \{d \in \mathcal{C} : \mathrm{rel}(q, d) = 1\}$. Un *sistema de recuperación ordenada* asigna a cada documento una puntuación $s(q, d) \in \mathbb{R}$ y devuelve los documentos en orden decreciente de puntuación. Se denota por T_k el conjunto de los k primeros.

Los documentos se representan por los *términos* que contienen, tras una normalización (minúsculas, eliminación de signos y, a veces, de palabras vacías). Un *índice invertido* guarda, para cada término t, la lista de documentos que lo contienen con su frecuencia. Para puntuar una consulta basta recorrer las listas de sus términos, de modo que el coste es proporcional a la suma de las longitudes de esas listas y no a N [17, §11.1.4].

2.2 Frecuencia de término y frecuencia inversa de documento

Definición 2.2 (Pesos tf-idf). Sean $f_{t,d}$ el número de apariciones del término t en el documento d (*frecuencia de término*) y df_t el número de documentos de $\mathcal{C}$ que contienen t (*frecuencia de documento*). La *frecuencia inversa de documento* y el *peso tf-idf* son

$$\mathrm{idf}_t = \ln \frac{N}{\mathrm{df}_t}, \qquad w_{t,d} = f_{t,d} \cdot \mathrm{idf}_t \,.$$

Una variante frecuente sustituye $f_{t,d}$ por el peso sublineal $1 + \ln f_{t,d}$ si $f_{t,d} > 0$ y por 0 en otro caso [24, §6.4.1].

Manning *et al.* usan logaritmos decimales [24, §6.2.1]. La base no altera ningún ranking basado en el coseno, porque cambiarla multiplica todos los pesos por la misma constante positiva (proposición 2.5).

Observación 2.3 (La idf como información). Si se elige un documento de $\mathcal{C}$ al azar con probabilidad uniforme, la probabilidad de que contenga t es df_t/N, e $\mathrm{idf}_t = -\ln(\mathrm{df}_t/N)$ es la información propia de ese suceso, medida en nats (módulo 6). Un término presente en todos los documentos no aporta información sobre cuál es relevante y recibe $\mathrm{idf}_t = 0$; un término muy raro es muy informativo. El peso tf-idf es alto cuando el término es frecuente en el documento y raro en la colección.

2.3 El modelo de espacio vectorial

Definición 2.4 (Similitud coseno). Fijado un orden del vocabulario de términos, cada documento se representa por el vector $V(d) = (w_{t,d})_t$ y la consulta por $V(q)$, construido del mismo modo. La puntuación del *modelo de espacio vectorial* es

$$\cos(q,d) = \frac{V(q)\cdot V(d)}{\|V(q)\|\,\|V(d)\|} \qquad [24,\ §6.3].$$

Proposición 2.5 (Invariancia de escala). *Para todo $\lambda,\mu>0$, $\cos(\lambda a,\mu b)=\cos(a,b)$. En particular, el coseno no cambia si todos los pesos se multiplican por una constante ni si un documento se concatena consigo mismo.*

La normalización por la norma elimina así la ventaja de los documentos largos, que contienen más términos de la consulta por azar. Si los vectores se normalizan al indexar, el coseno se reduce a un producto escalar, y puntuar toda la colección es un producto matriz por vector.

2.4 El principio de ordenación por probabilidad

El modelo de espacio vectorial es geométrico y su justificación es empírica. El enfoque probabilístico parte de la pregunta que importa: ¿cuál es la probabilidad de que d sea relevante para q? [24, §11.2].

Teorema 2.7 (Principio de ordenación por probabilidad). *Sea $p_d = P(R=1 \mid d,q)$ la probabilidad de que d sea relevante y supóngase que la relevancia de cada documento no depende de qué otros documentos se muestren. Para cada k, entre todos los conjuntos $S\subseteq\mathcal{C}$ con $|S|=k$, el número esperado de documentos relevantes $\mathbb{E}\big[|S\cap\mathcal{R}_q|\big]$ es máximo cuando S está formado por los k documentos de mayor p_d. Por tanto, ordenar por p_d maximiza a la vez la precisión esperada en todos los cortes k.*

La hipótesis de independencia entre documentos es restrictiva: ignora, por ejemplo, que dos duplicados aportan la misma información. Como $x\mapsto x/(1-x)$ es creciente en $[0,1)$, ordenar por p_d equivale a ordenar por las *razones de probabilidades* $O(R\mid d,q) = p_d/(1-p_d)$, que es lo que estima el modelo siguiente.

2.5 El modelo de independencia binaria

Definición 2.8 (Modelo de independencia binaria). Cada documento se representa por el vector binario $x=(x_t)_t$ con $x_t = \mathbb{1}[t\in d]$. Para cada término t de la consulta se definen $p_t = P(x_t=1\mid R=1,q)$ y $u_t = P(x_t=1\mid R=0,q)$. El modelo supone que los x_t son independientes condicionalmente a la relevancia y que los términos que no están en la consulta tienen la misma distribución en documentos relevantes y no relevantes [24, §11.3].

Proposición 2.9 (Valor de estado de recuperación). *Bajo el modelo de independencia binaria, ordenar por $O(R \mid x, q)$ equivale a ordenar por*

$$\mathrm{RSV}(d) = \sum_{t \in q \cap d} c_t, \qquad c_t = \ln \frac{p_t\,(1-u_t)}{u_t\,(1-p_t)}.$$

Corolario 2.10 (Sin juicios de relevancia). *Si se toma $p_t = 1/2$ y se estima u_t por $\mathrm{df}_t\,/N$, suponiendo que casi todos los documentos son no relevantes, entonces $c_t = \ln\big((N - \mathrm{df}_t)/\,\mathrm{df}_t\big)$, que se aproxima a idf_t cuando $\mathrm{df}_t \ll N$.*

El modelo probabilístico recupera así la idf como peso de un término. Al suavizar las estimaciones sumando $1/2$ a los conteos se obtiene el *peso de Robertson–Spärck Jones*, $\ln\big((N - \mathrm{df}_t + 0{,}5)/(\mathrm{df}_t + 0{,}5)\big)$ [30], que es negativo cuando $\mathrm{df}_t > N/2$: con $N = 10$ y $\mathrm{df}_t = 8$ vale $\ln(2{,}5/8{,}5) = -1{,}2238$. Para evitar pesos negativos, muchas implementaciones usan la variante $\ln\big(1 + (N - \mathrm{df}_t + 0{,}5)/(\mathrm{df}_t + 0{,}5)\big)$, que en ese caso vale 0,2578; es la que se adopta en estos apuntes.

2.6 BM25

El modelo binario ignora cuántas veces aparece un término y la longitud del documento. La función BM25 incorpora ambas cosas con dos principios: la frecuencia *se satura*, de modo que la décima aparición aporta menos que la primera, y la longitud *se normaliza*, porque un documento largo contiene más apariciones por azar [24, §11.4.3], [30].

Definición 2.11 (BM25). Sean $k_1 \geq 0$ y $0 \leq b \leq 1$, $n_t = \mathrm{df}_t$, $\bar{\ell}$ la longitud media de los documentos y $\ell = |d|/\bar{\ell}$. La puntuación BM25 es

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{idf}_t^{\mathrm{BM}}\; g(f_{t,d}, \ell), \qquad g(f, \ell) = \frac{f\,(k_1 + 1)}{f + k_1\,(1 - b + b\,\ell)},$$

con $\mathrm{idf}_t^{\mathrm{BM}} = \ln\big(1 + (N - n_t + 0{,}5)/(n_t + 0{,}5)\big)$. Los valores habituales son $k_1 \in [1{,}2;\ 2]$ y $b = 0{,}75$.

Proposición 2.12 (Saturación y normalización). *Sean $k_1 > 0$ y $0 \leq b \leq 1$. El factor $g(f, \ell)$ es estrictamente creciente en f, tiende a $k_1 + 1$ cuando $f \to \infty$ y, si $b > 0$, es estrictamente decreciente en ℓ para $f > 0$. Además, $g(1, 1) = 1$; con $b = 0$ no depende de ℓ, y con $k_1 = 0$ vale 1 para todo $f > 0$, de modo que BM25 se reduce a sumar los pesos idf de los términos presentes.*

La demostración de las tres primeras afirmaciones se pide en el ejercicio 2.6; las restantes se comprueban sustituyendo. El parámetro k_1 controla la rapidez de la saturación y b la intensidad de la normalización por longitud: con $b = 1$ la frecuencia se mide en relación con la longitud relativa del documento. Robertson y Zaragoza justifican la forma de g como aproximación de un modelo de dos poissonianas, en el que cada término es o no «elite» para el documento [30].

Observación 2.15. BM25 sigue siendo una referencia difícil de batir en muchas colecciones y es la parte léxica habitual de los sistemas de búsqueda híbrida (tema 4). Acierta con nombres propios, códigos y términos poco frecuentes, en los que la coincidencia exacta es lo que importa, y falla con sinónimos y paráfrasis: «coche» y «automóvil» no comparten ningún término.

Tema 3

Evaluación de la recuperación y de los modelos

Para elegir entre dos recuperadores, o entre dos modelos de lenguaje, hace falta medir. Este tema define las métricas clásicas de la recuperación de información, primero para conjuntos de resultados y después para rankings, demuestra sus propiedades básicas y pasa a la evaluación de modelos de lenguaje con pruebas de referencia (*benchmarks*): errores estándar, el estimador insesgado de pass@k, la contaminación de los datos y los jueces automáticos. Se siguen el capítulo 8 de Manning, Raghavan y Schütze [24], la §11.2 de Jurafsky y Martin [17], la clase 11 de CS224N [38] y los artículos de Chen *et al.* [6], Brown *et al.* [5] y Zheng *et al.* [42].

3.1 Conjuntos de resultados

Definición 3.1 (Precisión, exhaustividad y medida F). Sea S el conjunto de documentos devueltos para una consulta y $\mathcal{R}$ el de relevantes, con $S, \mathcal{R} \neq \varnothing$. Se definen la *precisión* y la *exhaustividad* (*recall*)

$$P = \frac{|S \cap \mathcal{R}|}{|S|}, \qquad R = \frac{|S \cap \mathcal{R}|}{|\mathcal{R}|},$$

y, para $\beta > 0$, la *medida* $F_\beta = (1+\beta^2)PR/(\beta^2 P + R)$, que para $\beta = 1$ es la media armónica $F_1 = 2PR/(P+R)$ [24, §8.3]. Si $P = R = 0$ se toma $F_\beta = 0$.

Proposición 3.2 (Propiedades de F_1). *Si $P + R > 0$: (i) $F_1 = 2|S \cap \mathcal{R}|/(|S| + |\mathcal{R}|)$; (ii) $\min(P, R) \leq F_1 \leq (P+R)/2$, con igualdad en ambos lados si y solo si $P = R$.*

La media armónica queda cerca del menor de los dos valores. Un sistema que devuelve la colección entera tiene $R = 1$ y una precisión mínima; su media aritmética sería mayor que $1/2$, pero su F_1 es casi nula (ejercicio 3.2).

3.2 Rankings

Definición 3.3 (Métricas de un ranking). Sea un ranking con T_k sus k primeros documentos y sea $\mathcal{R}$, con $|\mathcal{R}| > 0$, el conjunto de relevantes.

(i) *Precisión* y *exhaustividad en el corte* k: $\mathrm{P@}\,k = |T_k \cap \mathcal{R}|/k$ y $\mathrm{R@}\,k = |T_k \cap \mathcal{R}|/|\mathcal{R}|$.

(ii) *Precisión media*: si $k_1 < k_2 < \dots$ son las posiciones de los relevantes recuperados, $\mathrm{AP} = \frac{1}{|\mathcal{R}|}\sum_i \mathrm{P@}\,k_i$; los relevantes no recuperados aportan 0. La media de AP sobre un conjunto de consultas es la MAP.

(iii) *R-precisión*: $\mathrm{P@}|\mathcal{R}|$.

(iv) *Rango recíproco*: $\mathrm{RR} = 1/k_1$, o 0 si no se recupera ningún relevante; su media sobre las consultas es el MRR.

(v) *Ganancia acumulada descontada normalizada*: con relevancias graduadas $g_m \geq 0$ en la posición m,

$$\mathrm{nDCG@}\,k = \frac{1}{Z_k} \sum_{m=1}^{k} \frac{2^{g_m} - 1}{\log_2(1+m)},$$

donde Z_k es el valor del sumatorio para el orden ideal, de modo que el ranking perfecto obtiene 1 [24, §8.4].

Proposición 3.4 (Monotonía). $\mathrm{R@}\,k$ *es no decreciente en* k*. En cambio,* $\mathrm{P@}\,k$ *puede decrecer.*

Observación 3.7 (Qué métrica importa en RAG)*.* En un sistema RAG, los k trozos recuperados se añaden al *prompt*. Si el trozo que contiene la respuesta no está en T_k, el generador no puede usarlo; por eso la métrica más relevante es $\mathrm{R@}\,k$ para el k que se usará, y el orden dentro de T_k importa menos. En la práctica del módulo, sobre 21 apuntes y 8 consultas, un recuperador de trigramas de caracteres obtiene $\mathrm{P@}\,1 = 1{,}000$ y tf-idf y BM25 obtienen 0,875: la diferencia es una sola consulta, y con tan pocas no hay base para preferir uno (apartado 3.3).

3.3 Evaluación de modelos de lenguaje

Un *benchmark* es un conjunto de tareas con respuesta comprobable. La cuadro 3.1 recoge cuatro de los más citados.

Benchmark	Qué mide	Cómo se puntúa
MMLU [12]	Conocimientos en 57 materias	Pregunta de cuatro opciones
GSM8K [7]	Problemas aritméticos de enunciado	Respuesta numérica exacta
HumanEval [6]	Escritura de funciones en Python	Superar pruebas unitarias (pass@k)
SWE-bench [15]	Corrección de incidencias reales de GitHub	Superar las pruebas del repositorio

Cuadro 3.1: Cuatro *benchmarks* frecuentes en los informes de modelos.

Proposición 3.8 (Error estándar de una exactitud). *Sean* $X_1, \ldots, X_n$ *los indicadores de acierto en* n *preguntas, independientes y de Bernoulli de parámetro* p*, y* $\hat{p} = \bar{X}$*. Entonces* $\mathbb{E}[\hat{p}] = p$ *y* $\mathrm{Var}(\hat{p}) = p(1-p)/n$*. El* error estándar *estimado es* $\mathrm{SE} = \sqrt{\hat{p}(1-\hat{p})/n}$ *y, por el teorema central del límite,* $\hat{p} \pm 1{,}96\ \mathrm{SE}$ *es un intervalo de confianza aproximado del 95 %.*

Si dos modelos se evalúan sobre muestras independientes, la varianza de la diferencia de exactitudes es la suma de las varianzas. Si se evalúan sobre las mismas preguntas, $\mathrm{Var}(\hat{p}_A - \hat{p}_B) = \mathrm{Var}(\hat{p}_A) + \mathrm{Var}(\hat{p}_B) - 2\,\mathrm{Cov}(\hat{p}_A, \hat{p}_B)$, y la correlación positiva entre los aciertos de ambos reduce el error; por eso conviene un análisis emparejado pregunta a pregunta.

Definición 3.10 (pass@k). Para un problema de programación, sea p la probabilidad de que una solución generada supere las pruebas. La métrica *pass@k* es la probabilidad de que al menos una de k soluciones independientes las supere, $1 - (1-p)^k$, promediada sobre los problemas [6, §2.1].

Proposición 3.11 (Estimador insesgado de pass@k). *Se generan $n \geq k$ soluciones independientes de un problema y c de ellas superan las pruebas. Entonces*

$$\widehat{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

es un estimador insesgado de $1-(1-p)^k$, con el convenio $\binom{a}{k} = 0$ si $a < k$.

La demostración se pide en el ejercicio 3.7. Estimar p por c/n y sustituir en $1-(1-p)^k$ da un estimador sesgado: con $n = 10$, $c = 3$ y $k = 2$, el estimador insesgado vale $1 - \binom{7}{2}/\binom{10}{2} = 1 - 21/45 = 0{,}5333$, y la sustitución, $1 - 0{,}7^2 = 0{,}51$.

3.4 Contaminación, saturación y jueces

Definición 3.12 (Contaminación). Un *benchmark* está *contaminado* para un modelo si alguno de sus ejemplos de prueba, o un texto muy próximo, aparece en los datos de entrenamiento. Brown *et al.* marcaron como potencialmente contaminado todo ejemplo que comparte al menos un 13-grama con los datos de entrenamiento y compararon la exactitud en los ejemplos limpios y en los marcados [5, §4].

Un ejemplo de L tokens contiene $L - 12$ 13-gramas, de modo que la prueba consiste en buscar cada uno en un índice de los 13-gramas del corpus. Otras señales de contaminación son que la exactitud caiga con preguntas reformuladas, o que sea mejor en problemas publicados antes de la fecha de corte de los datos que en los posteriores. Por eso surgen benchmarks con preguntas nuevas, privadas o renovadas periódicamente.

Observación 3.13 (Saturación y ley de Goodhart). Cuando todos los modelos superan el 90 % en un benchmark, este deja de discriminar entre ellos. Y si se ajustan las decisiones de desarrollo mirando el conjunto de prueba, la nota sube sin que suba la capacidad: una medida convertida en objetivo deja de ser una buena medida.

Para respuestas abiertas no existe una respuesta exacta con la que comparar. Se usan dos tipos de jueces. Las *comparaciones humanas por pares*, como las de Chatbot Arena, agregan los votos con el modelo de Bradley–Terry del módulo 21. El *LLM como juez* pide a un modelo fuerte que puntúe o compare respuestas [42]. Zheng *et al.* documentan tres sesgos de estos jueces: preferencia por la respuesta mostrada en primer lugar (sesgo de posición), por las respuestas largas (sesgo de verbosidad) y, a veces, por las generadas por el propio juez. Se mitigan evaluando cada par en los dos órdenes y contando como empate un veredicto que cambia, con rúbricas explícitas y validando la concordancia con jueces humanos [42].

Tema 4

Recuperación densa y búsqueda de vecinos más próximos

Los modelos léxicos del tema 2 solo comparan términos idénticos. La recuperación densa representa consultas y pasajes por vectores de dimensión fija aprendidos por una red, de modo que textos con significado parecido quedan próximos aunque no compartan palabras. Este tema formula los codificadores duales y su entrenamiento contrastivo, relaciona las tres medidas de proximidad habituales (coseno, producto escalar y distancia euclídea) y estudia cómo buscar entre millones de vectores: la cuantización de producto, los índices invertidos, los grafos HNSW y la combinación con la búsqueda léxica. Se siguen la §11.3 de Jurafsky y Martin [17], los artículos de Karpukhin *et al.* [19], Jégou, Douze y Schmid [14], Malkov y Yashunin [23] y Johnson, Douze y Jégou [16], y la clase 10 de CS224N [38].

4.1 Codificadores duales

Definición 4.1 (Codificador dual). Un *codificador dual* consta de dos redes E_Q y E_P que asignan a cada consulta q y a cada pasaje p vectores de $\mathbb{R}^d$. La similitud es el producto escalar

$$s(q,p) = E_Q(q)^\top E_P(p).$$

Los vectores $E_P(p)$ de toda la colección se calculan una sola vez, al indexar; ante cada consulta solo se calcula $E_Q(q)$.

En la recuperación densa de pasajes (DPR) de Karpukhin *et al.*, E_Q y E_P son dos codificadores BERT independientes y el vector es el estado del token `[CLS]`, con $d = 768$ [19]. La separación en dos redes es lo que permite precalcular el índice: un *codificador cruzado*, que procesa la consulta y el pasaje juntos, suele ser más preciso, pero exige una pasada de la red por cada pasaje candidato, por lo que se reserva para reordenar unos pocos resultados [17, §11.3].

Definición 4.2 (Pérdida contrastiva). Dada una consulta q con un pasaje relevante p^+ y r pasajes no relevantes $p_1^-, \dots, p_r^-$, con puntuaciones $s^+ = s(q,p^+)$ y $s_j^- = s(q,p_j^-)$, la pérdida es la log-verosimilitud negativa del positivo bajo una softmax sobre los $r+1$ candidatos:

$$\mathcal{L} = -\ln \frac{e^{s^+}}{e^{s^+} + \sum_{j=1}^{r} e^{s_j^-}}.$$

Proposición 4.3 (Gradiente respecto de las puntuaciones). *Sea π la softmax de las $r+1$ puntuaciones. Entonces $\partial\mathcal{L}/\partial s^+ = \pi^+ - 1$ y $\partial\mathcal{L}/\partial s_j^- = \pi_j^-$.*

El entrenamiento sube la puntuación del positivo y baja la de cada negativo en proporción a la probabilidad que el modelo le asigna: los negativos difíciles, con puntuación alta, reciben el mayor empuje.

Definición 4.5 (Negativos del lote). En un lote de B pares (q_i, p_i^+), sean $Q \in \mathbb{R}^{B\times d}$ y $P \in \mathbb{R}^{B\times d}$ las matrices de vectores de consultas y de pasajes positivos, y $S = QP^\top$. Para la consulta i, el positivo es p_i^+ y los negativos son los positivos de las demás consultas: la pérdida del lote es la media de $-\ln \operatorname{softmax}(S_{i,:})_i$.

Con $2B$ codificaciones se obtienen B^2 puntuaciones y $B-1$ negativos por consulta sin coste adicional de codificación. Karpukhin *et al.* añaden a cada consulta un negativo difícil, el pasaje mejor puntuado por BM25 que no contiene la respuesta [19].

4.2 Coseno, producto escalar y distancia euclídea

Teorema 4.6. *Para $a, b \in \mathbb{R}^d$, $\|a-b\|^2 = \|a\|^2 + \|b\|^2 - 2a^\top b$. En particular, si $\|a\| = \|b\| = 1$, entonces $\|a-b\|^2 = 2 - 2\cos(a,b)$.*

Con vectores normalizados, mayor coseno equivale a menor distancia, de modo que las estructuras diseñadas para buscar vecinos próximos en distancia euclídea sirven también para el coseno (ejercicio 4.6). Con $a = (0{,}6;\ 0{,}8)$ y $b = (1, 0)$: $\|a-b\|^2 = 0{,}16+0{,}64 = 0{,}8 = 2-2\cdot 0{,}6$. Si los vectores no están normalizados, buscar el máximo producto escalar (MIPS) no equivale a buscar el vecino más próximo, pero el problema se reduce a él.

Proposición 4.7 (Reducción de MIPS a vecino más próximo). *Sean $x_1, \dots, x_N \in \mathbb{R}^d$, $M \geq \max_i \|x_i\|$ y los vectores aumentados $\tilde{x}_i = \left(x_i, \sqrt{M^2 - \|x_i\|^2}\right) \in \mathbb{R}^{d+1}$ y $\tilde{q} = (q, 0)$. Entonces $\|\tilde{q} - \tilde{x}_i\|^2 = \|q\|^2 + M^2 - 2q^\top x_i$, de modo que el vecino más próximo de $\tilde{q}$ entre los $\tilde{x}_i$ es el x_i de máximo producto escalar con q.*

La demostración se pide en el ejercicio 4.7. Con $x_1 = (3, 0)$, $x_2 = (1, 1)$, $x_3 = (0, 2)$, $q = (1, 1)$ y $M = 3$, los productos escalares son 3, 2 y 2, y las distancias al cuadrado de los vectores aumentados, $2 + 9 - 6 = 5$, 7 y 7: el orden se conserva.

4.3 Búsqueda exacta y aproximada

La búsqueda exacta de los k vectores más próximos recorre toda la colección: N productos escalares de dimensión d, es decir, Nd multiplicaciones y sumas por consulta. Con $N = 10^7$ pasajes y $d = 768$ son $7{,}68 \cdot 10^9$ por consulta, y el índice ocupa 30,7 GB en coma flotante de 32 bits. La *búsqueda aproximada* renuncia a la exactitud a cambio de velocidad y memoria; su calidad se mide con la exhaustividad respecto de los k vecinos exactos.

4.3.1 Cuantización de producto

Definición 4.8 (Cuantización de producto). Sea $d = mD$. Cada vector $x \in \mathbb{R}^d$ se divide en m subvectores consecutivos $x^{(1)}, \dots, x^{(m)} \in \mathbb{R}^D$. Para cada j se aprende con k-medias un libro de códigos $\{c_{j,1}, \dots, c_{j,k^*}\} \subset \mathbb{R}^D$ sobre los subvectores j-ésimos de la colección. El *código* de x es $(i_1, \dots, i_m)$, con i_j el índice del centroide más próximo a $x^{(j)}$, y su reconstrucción es $\hat{x} = (c_{1,i_1}, \dots, c_{m,i_m})$ [14].

Proposición 4.9 (Coste de la cuantización de producto). *(i) Cada código ocupa $m \log_2 k^*$ bits, los libros de códigos ocupan $k^* d$ números y el número de reconstrucciones distintas es $(k^*)^m$. (ii) Para una consulta y, la distancia asimétrica $\|y - \hat{x}\|^2$ se descompone*

como

$$\|y - \hat{x}\|^2 = \sum_{j=1}^{m} \|y^{(j)} - c_{j,i_j}\|^2,$$

de modo que, precalculadas las m tablas de k^* distancias $\|y^{(j)} - c_{j,i}\|^2$, con unas $k^* d$ operaciones, cada vector de la colección se evalúa con m consultas a tabla y $m - 1$ sumas.

Para no recorrer toda la colección, se añade un *índice invertido*: un cuantizador grueso de k' celdas asigna cada vector a su celda y la búsqueda solo explora las w celdas más próximas a la consulta, una fracción del orden de w/k' de la colección [14]. La biblioteca FAISS implementa estas estructuras, también en GPU [16].

4.3.2 Grafos de mundo pequeño navegables y jerárquicos

Definición 4.11 (HNSW). Un índice HNSW es una jerarquía de grafos sobre los vectores de la colección. Al insertar un elemento se le asigna un nivel máximo $\lambda = \lfloor -\ln(U)\, m_L \rfloor$, con U uniforme en $(0, 1]$ y $m_L > 0$, y se inserta en las capas $0, \ldots, \lambda$, unido en cada una a unos M vecinos próximos. La búsqueda empieza en un punto de entrada de la capa superior, avanza de forma voraz hacia el vecino más próximo a la consulta en cada capa, usa el resultado como punto de entrada de la capa inferior y, en la capa 0, mantiene una lista de ef candidatos [23].

Proposición 4.12 (Distribución de los niveles). *Para cada entero $j \geq 0$, $P(\lambda \geq j) = e^{-j/m_L}$. Con $m_L = 1/\ln M$, $P(\lambda \geq j) = M^{-j}$, el número esperado de elementos en la capa j es $N M^{-j}$ y la capa superior esperada está en torno a $\log_M N$.*

Cada capa tiene unas M veces menos elementos que la inferior, como los niveles de una lista con saltos: las capas altas, con enlaces largos, acercan rápidamente la búsqueda a la región de la consulta, y las bajas la refinan. Malkov y Yashunin observan experimentalmente un coste de búsqueda que crece de forma logarítmica con N [23]; no es un teorema para datos arbitrarios. Con $M = 16$ y $N = 10^6$, las capas 1 a 5 tienen en media 62 500, 3906, 244, 15 y 1 elementos.

4.4 Búsqueda híbrida

Los recuperadores densos y los léxicos fallan en casos distintos. Un recuperador denso sabe que «coche» y «automóvil» se parecen; BM25 no. BM25 acierta con nombres propios, códigos de error o identificadores, que un recuperador denso, entrenado en otro dominio, puede confundir con otros parecidos. Los sistemas de búsqueda híbrida combinan los rankings de ambos.

Definición 4.13 (Fusión por rango recíproco). Sean varios rankings $r = 1, \ldots, R$ y $\mathrm{rank}_r(d)$ la posición de d en el ranking r. La puntuación fusionada es

$$\mathrm{RRF}(d) = \sum_{r:\, d \in r} \frac{1}{\kappa + \mathrm{rank}_r(d)},$$

con $\kappa = 60$ en la propuesta original de Cormack, Clarke y Büttcher [8].

Tema 5

Generación aumentada por recuperación

La generación aumentada por recuperación (RAG, del inglés *retrieval-augmented generation*) combina un recuperador, que selecciona fragmentos de una colección, con un modelo de lenguaje que genera la respuesta condicionado a ellos. Este tema formula los dos modelos probabilísticos de Lewis *et al.*, estudia las decisiones de diseño de un sistema práctico (troceo y construcción del *prompt*) y su evaluación, separando el error del recuperador del error del generador. Se siguen el artículo de Lewis *et al.* [21], las §11.4 y §11.6 de Jurafsky y Martin [17], la clase 10 de CS224N [38], la tarea de OpenQA de CS224U [28] y la segunda tarea de 11-711 de Carnegie Mellon [25].

5.1 Memoria paramétrica y no paramétrica

Un modelo de lenguaje guarda conocimiento en sus pesos: es una *memoria paramétrica*, que no se puede consultar ni actualizar sin reentrenar y que no indica de dónde procede cada dato. Una colección de documentos indexada es una *memoria no paramétrica*: se actualiza añadiendo documentos y cada fragmento recuperado tiene una fuente identificable. RAG usa las dos [21]. El esquema básico, «recuperar y leer», tiene una fase de indexado, que se hace una vez, y una fase de consulta, que se repite con cada pregunta (figura 5.1) [17, §11.4].

5.2 Los modelos RAG-Sequence y RAG-Token

Lewis *et al.* tratan el documento recuperado z como una variable latente [21, §2.1]. El recuperador es un codificador dual (definición 4.1) que define $p_\eta(z \mid x) \propto \exp\big(E_P(z)^\top E_Q(x)\big)$, restringida en la práctica a los k documentos de mayor puntuación y renormalizada sobre ellos. El generador $p_\theta(y_i \mid x, z, y_{<i})$ es un modelo de secuencia a secuencia que recibe la pregunta y el documento.

Definición 5.1 (RAG-Sequence y RAG-Token). Sea $Z_k(x)$ el conjunto de los k documentos recuperados para x y sea $y = y_{1:n}$ la respuesta.

(i) En *RAG-Sequence*, un mismo documento se usa para toda la respuesta:

$$p_{\text{seq}}(y \mid x) = \sum_{z \in Z_k(x)} p_\eta(z \mid x) \prod_{i=1}^{n} p_\theta(y_i \mid x, z, y_{<i}).$$

(ii) En *RAG-Token*, cada token puede apoyarse en un documento distinto:

$$p_{\text{tok}}(y \mid x) = \prod_{i=1}^{n} \sum_{z \in Z_k(x)} p_\eta(z \mid x)\, p_\theta(y_i \mid x, z, y_{<i}).$$

Proposición 5.2. *Para cada longitud n fija, $p_{\text{seq}}(\cdot \mid x)$ y $p_{\text{tok}}(\cdot \mid x)$ son distribuciones de probabilidad sobre $\mathcal{V}^n$. Coinciden si $n = 1$ o si $p_\eta(\cdot \mid x)$ concentra toda la masa en un único documento.*

La demostración se pide en el ejercicio 5.7. RAG-Sequence es una mezcla de k modelos de lenguaje, uno por documento, con pesos $p_\eta(z \mid x)$. RAG-Token mezcla en cada paso las distribuciones del token siguiente; puede, por ejemplo, tomar el título de un libro de un documento y el nombre del autor de otro.

Observación 5.4 (Entrenamiento). Lewis *et al.* entrenan conjuntamente el codificador de consultas y el generador minimizando $-\ln p(y \mid x)$ sobre pares de pregunta y respuesta, sin indicar qué documento es el bueno: el gradiente llega al recuperador a través de los pesos $p_\eta(z \mid x)$ de la mezcla. El codificador de documentos y el índice se mantienen fijos, porque actualizarlos obligaría a recodificar toda la colección [21, §2.4]. En los sistemas actuales con modelos de propósito general es habitual no entrenar nada: el recuperador y el modelo se usan congelados y los fragmentos se insertan en el *prompt*.

5.3 Troceo

Definición 5.5 (Troceo con solape). Un documento de L tokens se divide en trozos de τ tokens con un solape de s tokens, $0 \le s < \tau$. El trozo $j = 0, 1, \ldots$ empieza en la posición $j(\tau - s)$; el paso entre inicios es $\tau - s$.

Proposición 5.6 (Número de trozos). *Si $L \ge \tau$, el número de trozos necesarios para cubrir el documento es*

$$n = \left\lceil \frac{L - \tau}{\tau - s} \right\rceil + 1,$$

y el número total de tokens indexados es aproximadamente $L\,\tau/(\tau - s)$.

Con $L = 1000$, $\tau = 200$ y $s = 50$, los inicios son $0, 150, \ldots, 900$ y hay $\lceil 800/150 \rceil + 1 = 7$ trozos. El solape evita cortar una idea por la mitad, a costa de indexar $\tau/(\tau - s) = 1{,}33$ veces más texto. Los trozos pequeños dan una recuperación más precisa, pero con poco contexto alrededor; los grandes, lo contrario, y además ocupan más espacio en el *prompt*.

5.4 El *prompt* y su coste

El *prompt* de un sistema RAG suele tener tres partes: instrucciones (responder solo con la información proporcionada, citar el fragmento usado y decir que no se sabe si la respuesta no está), los k fragmentos, cada uno delimitado por marcas con un identificador, y la pregunta. Las marcas separan lo que es material de consulta de lo que son instrucciones, una separación que reaparece en el tema 6 como defensa parcial frente a la inyección de instrucciones. Con 3 fragmentos de 200 tokens y 50 tokens de pregunta, el *prompt* tiene unos 650 tokens, frente a los 8000 de pegar todo el material (tema 1). Como los modelos aprovechan peor la información situada en mitad de contextos largos [22], conviene colocar los fragmentos mejor puntuados al principio o al final.

5.5 Evaluación de un sistema RAG

Un sistema RAG falla por dos motivos distintos: el recuperador no encuentra el fragmento con la respuesta o el generador no la extrae bien. Conviene medirlos por separado.

Proposición 5.7 (Descomposición del acierto). *Sea r la probabilidad de que algún fragmento de T_k contenga la respuesta, a_1 la probabilidad de que el generador acierte cuando la respuesta está en el contexto y a_0 cuando no está. La exactitud del sistema es*

$$\mathrm{Acc} = r\,a_1 + (1-r)\,a_0 \;\le\; r + (1-r)\,a_0.$$

La probabilidad a_0 corresponde a lo que el modelo sabe de memoria, el modo «de libro cerrado». La tarea de 11-711 pide precisamente comparar el sistema con recuperación con el mismo modelo sin ella [25]. Aumentar k sube r, porque $\mathrm{R@}\,k$ no decrece (proposición 3.4), pero puede bajar a_1 al llenar el contexto de fragmentos irrelevantes (ejercicio 5.6).

Definición 5.8 (Exactitud exacta y F_1 de tokens). Sea ν una normalización de cadenas: paso a minúsculas, eliminación de signos de puntuación y de artículos y unificación de espacios. Para una predicción $\hat{a}$ y una respuesta de referencia a, la *exactitud exacta* es $\mathrm{EM} = \mathbb{1}[\nu(\hat{a}) = \nu(a)]$. Si $\hat{T}$ y T son los multiconjuntos de tokens de $\nu(\hat{a})$ y $\nu(a)$ y $c = |\hat{T} \cap T|$ es el número de tokens comunes (contados con multiplicidad), la precisión es $c/|\hat{T}|$, la exhaustividad $c/|T|$ y la F_1 su media armónica, o 0 si $c = 0$. Con varias referencias se toma el máximo sobre ellas [29], [17, §11.6].

Son las métricas de SQuAD [29], que la tarea de 11-711 usa junto con la exhaustividad de la respuesta [25]; la normalización original elimina los artículos ingleses, y aquí se adapta a los españoles.

Observación 5.10 (Fidelidad al contexto). Una respuesta correcta puede no proceder del contexto, sino de la memoria paramétrica. Para seleccionar demostraciones de un *prompt* de pocos ejemplos con recuperación, la tarea de CS224U exige dos condiciones a la vez: que la respuesta del modelo coincida con la de referencia y que la respuesta de referencia aparezca literalmente en los pasajes recuperados [28]. Así, las demostraciones enseñan al modelo a responder a partir del contexto (ejercicio 5.5).

Tema 6

Agentes y uso de herramientas

Un modelo de lenguaje solo produce texto. Para que actúe sobre el mundo (buscar, calcular, ejecutar código, enviar un mensaje) hay que integrarlo en un sistema que interprete parte de ese texto como acciones, las ejecute y le devuelva el resultado. Este tema formaliza ese sistema como un agente en el sentido de la inteligencia artificial clásica y como una política sobre un proceso de decisión parcialmente observable, describe el protocolo de las llamadas a herramientas y dos métodos para aprenderlas o usarlas (Toolformer y ReAct), cuantifica la fiabilidad de una cadena de pasos y estudia el principal riesgo de seguridad, la inyección de instrucciones. Se siguen el capítulo 2 de Russell y Norvig [31], el capítulo 3 de Sutton y Barto [34], el artículo de Kaelbling, Littman y Cassandra [18], los artículos de Yao *et al.* [39], Schick *et al.* [32] y Greshake *et al.* [11], la clase 10 de CS224N [38] y el curso CS294-196 de Berkeley [33].

6.1 Agentes racionales

Definición 6.1 (Agente). Un *agente* percibe su entorno mediante sensores y actúa sobre él mediante actuadores. Su comportamiento queda descrito por la *función de agente*, que asigna una acción a cada secuencia de percepciones; el *programa de agente* es la implementación concreta de esa función. Un agente es *racional* si, para cada secuencia de percepciones, elige la acción que maximiza el valor esperado de una *medida de rendimiento* dada, según la información que proporcionan las percepciones y el conocimiento previo del agente [31, cap. 2].

Russell y Norvig describen un entorno de tarea por sus cuatro elementos PEAS (rendimiento, entorno, actuadores y sensores, por sus iniciales en inglés) y lo clasifican según varias propiedades: total o parcialmente observable, determinista o estocástico, episódico o secuencial, estático o dinámico, discreto o continuo, de un agente o multiagente, y conocido o desconocido [31, cap. 2].

6.2 El agente como proceso de decisión parcialmente observable

Definición 6.3 (Proceso de decisión de Markov parcialmente observable). Un POMDP es una tupla $(\mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma)$ con estados $\mathcal{S}$, acciones $\mathcal{A}$, probabilidades de transición $T(s' \mid s, a)$, recompensa $R(s, a)$, observaciones Ω, probabilidades de observación $O(o \mid s', a)$ y factor de descuento $\gamma \in [0, 1]$ [18]. El agente no observa el estado, sino la *historia* $h_t = (o_1, a_1, o_2, \ldots, a_{t-1}, o_t)$. Una *política* es una distribución $\pi(a_t \mid h_t)$. Si las observaciones revelan el estado, el POMDP es un proceso de decisión de Markov [34, cap. 3].

Un agente basado en un modelo de lenguaje encaja en esta definición: la historia es el

contexto (instrucciones, mensajes del usuario, llamadas a herramientas y sus resultados), la política es p_θ restringida a las acciones admisibles (una llamada a herramienta o una respuesta final) y las observaciones son los resultados de las herramientas. El estado incluye todo lo que el agente no ve: el contenido de una web, el estado de un repositorio, las intenciones del usuario.

Definición 6.4 (Estado de creencia). El *estado de creencia* es la distribución $b_t(s) = P(s_t = s \mid h_t)$ sobre el estado oculto, dada la historia.

Proposición 6.5 (Actualización de la creencia). *Si el agente tiene la creencia b, ejecuta a y observa o, la nueva creencia es*

$$b'(s') = \frac{O(o \mid s', a) \sum_s T(s' \mid s, a)\, b(s)}{\sum_{s''} O(o \mid s'', a) \sum_s T(s'' \mid s, a)\, b(s)}.$$

Kaelbling *et al.* demuestran que la creencia es un estadístico suficiente de la historia para decidir de forma óptima, de modo que un POMDP equivale a un proceso de decisión de Markov sobre creencias [18]. Un modelo de lenguaje no mantiene una creencia explícita, pero condiciona su decisión a la historia completa (o a la parte que cabe en la ventana de contexto), que contiene la misma información.

6.3 Llamadas a herramientas

Definición 6.7 (Herramienta y llamada). Una *herramienta* se describe por un nombre, una descripción en lenguaje natural y un esquema de argumentos, habitualmente en JSON Schema. Una *llamada* es un texto estructurado generado por el modelo que contiene el nombre de la herramienta, unos argumentos y un identificador. El *sistema* que rodea al modelo valida la llamada contra el esquema, ejecuta la herramienta y añade al contexto el *resultado*, asociado al identificador de la llamada.

El modelo nunca ejecuta nada: genera texto. Que ese texto sea una llamada bien formada en el momento oportuno es algo que se aprende en el ajuste supervisado y por refuerzo (módulos 20 y 21). El bucle del agente es el siguiente:

1. Construir el contexto con las instrucciones, las descripciones de las herramientas y la historia.
2. Generar con el modelo. Si la salida es una respuesta final, devolverla y terminar.
3. Si es una llamada, validar sus argumentos; si no son válidos, añadir un mensaje de error a la historia.
4. Ejecutar la herramienta con los permisos del sistema y añadir el resultado a la historia.
5. Si se alcanza el número máximo de pasos, terminar con un aviso; si no, volver al paso 2.

Las API públicas de los tres modelos comerciales que se mencionan en el curso siguen este esquema (cuadro 6.1). En la de Anthropic, cada herramienta tiene `name`, `description` e `input_schema`; cuando Claude decide usar una, la respuesta termina con `stop_reason: "tool_use"` y contiene bloques `tool_use` con `id`, `name` e `input`, y la aplicación responde con bloques `tool_result` que llevan el `tool_use_id` correspondiente; la documentación distingue las herramientas que ejecuta la aplicación de las que ejecuta Anthropic en sus servidores, como la búsqueda web [2].

	Definición	Llamada del modelo	Resultado
Claude [2]	`name`, `description`, `input_schema`	bloque `tool_use` (`id`, `name`, `input`)	bloque `tool_result` con `tool_use_id`
Gemini [10]	`name`, `description`, `parameters`	`function_call` (`id`, `name`, `arguments`)	`function_result` con `call_id`
Grok [37]	`name`, `description`, `parameters`	`function_call` (`call_id`, `name`, `arguments`)	`function_call_output` con `call_id`

Cuadro 6.1: Campos de las llamadas a herramientas en la documentación pública consultada en octubre de 2026. Los nombres pueden cambiar entre versiones de cada API.

Observación 6.9 (Model Context Protocol). En noviembre de 2024, Anthropic presentó el *Model Context Protocol* (MCP), un estándar abierto para conectar aplicaciones de LLM con fuentes de datos y herramientas sin integraciones específicas para cada una [3]. Según su especificación, las aplicaciones (*anfitriones*) contienen *clientes* que se conectan a *servidores* mediante mensajes JSON-RPC 2.0; los servidores ofrecen *recursos* (datos de contexto), *prompts* (plantillas) y *herramientas* (funciones que el modelo puede invocar). La especificación pide el consentimiento explícito del usuario antes de invocar una herramienta y que las descripciones de herramientas procedentes de servidores no fiables se traten como no fiables [4].

6.3.1 Toolformer: aprender cuándo llamar

Schick *et al.* enseñan a un modelo a insertar llamadas sin anotación humana: el propio modelo propone posibles llamadas en un corpus, se ejecutan y solo se conservan las que ayudan a predecir el texto que sigue [32].

Definición 6.10 (Criterio de filtrado de Toolformer). Sea una llamada c insertada en la posición i de un texto x, con resultado r. Para un prefijo z, sea

$$L_i(z) = -\sum_{j \geq i} w_{j-i} \ln p_\theta(x_j \mid z, x_{1:j-1}),$$

la pérdida ponderada de los tokens siguientes, con pesos $w_t = \tilde{w}_t / \sum_s \tilde{w}_s$ y $\tilde{w}_t = \max(0, 1 - 0{,}2t)$, que dan más importancia a los tokens próximos. Sean L_i^+ la pérdida con la llamada y su resultado como prefijo y L_i^- el mínimo entre la pérdida sin llamada y con la llamada pero sin resultado. La llamada se conserva si $L_i^- - L_i^+ \geq \tau_f$, para un umbral τ_f [32, §2].

El criterio exige que lo útil sea el *resultado*: comparar también con la llamada sin resultado descarta llamadas cuyo texto, por sí solo, ya facilita la predicción. Con las llamadas conservadas se ajusta el modelo, que aprende a emitirlas donde reducen la pérdida.

6.3.2 ReAct: razonar y actuar

Definición 6.11 (Trayectoria ReAct). En ReAct, la salida del modelo en cada paso es un *pensamiento* en lenguaje natural o una *acción*; tras cada acción, el sistema añade la *observación* correspondiente. Una trayectoria tiene la forma

$$(\text{pensamiento}_1,\ \text{acción}_1,\ \text{observación}_1,\ \text{pensamiento}_2,\ \ldots)$$

y termina con una acción de respuesta [39].

Los pensamientos no modifican el entorno: sirven para descomponer la tarea, seguir el progreso y reaccionar a resultados inesperados. Yao *et al.* comparan ReAct con el razonamiento sin acciones (cadena de pensamiento), que no puede comprobar hechos y tiende a inventarlos, y con la actuación sin pensamientos, que pierde el hilo en tareas largas [39]. La mayoría de los agentes actuales siguen esta alternancia, con el razonamiento a veces oculto al usuario.

6.4 Fiabilidad de una cadena de pasos

Proposición 6.12 (Composición de pasos). *Si cada paso de un agente tiene éxito con probabilidad p, de forma independiente, y la tarea exige n pasos correctos, la probabilidad de completarla es p^n, y el número de pasos hasta el primer fallo sigue una distribución geométrica de media $1/(1-p)$.*

Con $p = 0{,}95$ y $n = 10$, $p^n = 0{,}599$; con $p = 0{,}99$, 0,904. Con $p = 0{,}95$, el primer fallo llega en media en el paso 20. Si cada paso se *verifica* (por ejemplo, ejecutando las pruebas tras cada edición) y se permite un reintento independiente, la probabilidad de éxito por paso pasa a $1 - (1-p)^2 = 0{,}9975$ y la de completar diez pasos, a $0{,}9975^{10} = 0{,}975$. Por eso los agentes útiles comprueban sus resultados y pueden corregirse; la independencia de los reintentos es, de nuevo, una idealización.

6.5 Riesgos: inyección de instrucciones

Definición 6.13 (Inyección de instrucciones). Hay *inyección de instrucciones* cuando un texto que el sistema trata como dato contiene órdenes y el modelo las obedece. Es *directa* si la escribe el propio usuario para saltarse las reglas del sistema, e *indirecta* si llega en un contenido que el agente lee durante la tarea: una web, un correo, un documento recuperado o el resultado de una herramienta [11].

El problema es estructural. Para el Transformer, instrucciones y datos son una única secuencia de tokens, y no existe un canal separado que distinga «lo que pide el usuario» de «lo que dice una web». Greshake *et al.* muestran que cualquiera que pueda escribir en una fuente que el agente consulta puede intentar controlarlo [11]. Las defensas conocidas son todas parciales (cuadro 6.2).

Defensa	Qué hace	Límite
Marcar los datos	Delimitar el contenido externo y avisar de que no contiene órdenes	El modelo puede ignorar el aviso
Mínimo privilegio	Lista blanca de herramientas y permisos por tarea	No impide respuestas engañosas
Confirmación humana	Pedir permiso antes de acciones irreversibles	Fatiga: se acepta sin leer
Entrenamiento	Ajuste con ejemplos de ataques	Reduce la tasa de éxito, no la anula

Cuadro 6.2: Defensas frente a la inyección de instrucciones y sus límites.

Observación 6.14 (Diseñar para el peor caso). El principio de diseño es suponer que el modelo *puede* ser engañado y acotar lo que un modelo engañado puede hacer. Si un ataque tiene éxito sobre el modelo con probabilidad 0,1 y el usuario aprueba sin leer una acción con

probabilidad 0,2, una acción dañina que requiere confirmación ocurre con probabilidad 0,02, si ambos sucesos son independientes; si la acción no está en la lista blanca, la probabilidad es nula, porque el sistema la impide aunque el modelo la solicite. La seguridad debe estar en el sistema, no en el *prompt*.

Bibliografía

[1] Anthropic. The Claude 3 model family: Opus, Sonnet, Haiku. Ficha técnica del modelo, marzo de 2024. https://www.anthropic.com/news/claude-3-family

[2] Anthropic. Tool use with Claude (documentación de la API). Consultada en octubre de 2026. https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview

[3] Anthropic. Introducing the Model Context Protocol. Anuncio del 25 de noviembre de 2024. https://www.anthropic.com/news/model-context-protocol

[4] Model Context Protocol. Specification, versión 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18

[5] T. B. Brown *et al.* Language models are few-shot learners. En *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. https://arxiv.org/abs/2005.14165

[6] M. Chen *et al.* Evaluating large language models trained on code. arXiv:2107.03374, 2021. https://arxiv.org/abs/2107.03374

[7] K. Cobbe *et al.* Training verifiers to solve math word problems. arXiv:2110.14168, 2021. https://arxiv.org/abs/2110.14168

[8] G. V. Cormack, C. L. A. Clarke y S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. En *Proceedings of SIGIR*, 2009.

[9] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530, 2024. https://arxiv.org/abs/2403.05530

[10] Google. Function calling with the Gemini API (documentación). Consultada en octubre de 2026. https://ai.google.dev/gemini-api/docs/function-calling

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz y M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv:2302.12173, 2023. https://arxiv.org/abs/2302.12173

[12] D. Hendrycks *et al.* Measuring massive multitask language understanding. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2009.03300

[13] W. Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[14] H. Jégou, M. Douze y C. Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[15] C. E. Jimenez *et al.* SWE-bench: Can language models resolve real-world GitHub issues? En *International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2310.06770

[16] J. Johnson, M. Douze y H. Jégou. Billion-scale similarity search with GPUs. arXiv:1702.08734, 2017. https://arxiv.org/abs/1702.08734

[17] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 11, «Information Retrieval and Retrieval-Augmented Generation». https://web.stanford.edu/~jurafsky/slp3/

[18] L. P. Kaelbling, M. L. Littman y A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[19] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen y W. Yih. Dense passage retrieval for open-domain question answering. En *Proceedings of EMNLP*, 2020. https://arxiv.org/abs/2004.04906

[20] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo e Y. Iwasawa. Large language models are zero-shot reasoners. En *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. https://arxiv.org/abs/2205.11916

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel y D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. En *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. https://arxiv.org/abs/2005.11401

[22] N. F. Liu *et al.* Lost in the middle: How language models use long contexts. *Transactions of the ACL*, 12, 2024. https://arxiv.org/abs/2307.03172

[23] Y. A. Malkov y D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. https://arxiv.org/abs/1603.09320

[24] C. D. Manning, P. Raghavan y H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Edición en línea de 2009, texto del curso CS276 de Stanford. Caps. 6 «Scoring, term weighting and the vector space model», 8 «Evaluation in information retrieval» y 11 «Probabilistic information retrieval». https://nlp.stanford.edu/IR-book/

[25] G. Neubig. 11-711: Advanced Natural Language Processing. Carnegie Mellon University, primavera de 2024. Tarea 2, «End-to-end NLP System Building». https://github.com/neubig/nlp-from-scratch-assignment-spring2024

[26] C. Olsson, N. Elhage, N. Nanda *et al.* In-context learning and induction heads. *Transformer Circuits Thread*, Anthropic, 2022. https://arxiv.org/abs/2209.11895

[27] J. von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov y M. Vladymyrov. Transformers learn in-context by gradient descent. En *Proceedings of ICML*, 2023. https://arxiv.org/abs/2212.07677

[28] C. Potts. CS224U: Natural Language Understanding. Stanford University, primavera de 2023. Unidad «Retrieval-augmented in-context learning» y tarea «Few-shot OpenQA with DSPy». https://web.stanford.edu/class/cs224u/

[29] P. Rajpurkar, J. Zhang, K. Lopyrev y P. Liang. SQuAD: 100,000+ questions for machine comprehension of text. En *Proceedings of EMNLP*, 2016. https://arxiv.org/abs/1606.05250

[30] S. Robertson y H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[31] S. Russell y P. Norvig. *Artificial Intelligence: A Modern Approach*, 4.ª ed. Pearson, 2021. Cap. 2, «Intelligent Agents». Ejercicios en https://github.com/aimacode/aima-exercises.

[32] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda y T. Scialom. Toolformer: Language models can teach themselves to use tools. En *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. https://arxiv.org/abs/2302.04761

[33] D. Song. CS294-196: Large Language Model Agents. University of California, Berkeley, otoño de 2024 (también como curso abierto en línea). https://classes.berkeley.edu/content/2024-fall-compsci-294-196-lec-196

[34] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*, 2.ª ed. MIT Press, 2018. Cap. 3, «Finite Markov Decision Processes». http://incompleteideas.net/book/the-book-2nd.html

[35] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery y D. Zhou. Self-consistency improves chain of thought reasoning in language models. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2203.11171

[36] J. Wei *et al.* Chain-of-thought prompting elicits reasoning in large language models. En *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. https://arxiv.org/abs/2201.11903

[37] xAI. Function calling (documentación de la API de Grok). Consultada en octubre de 2026. https://docs.x.ai/docs/guides/function-calling

[38] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clases 9 «Efficient Adaptation (Prompting + PEFT)», 10 «Agents, Tool Use, and RAG» y 11 «Benchmarking and Evaluation». https://web.stanford.edu/class/cs224n/

[39] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan e Y. Cao. ReAct: Synergizing reasoning and acting in language models. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2210.03629

[40] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao y K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. En *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. https://arxiv.org/abs/2305.10601

[41] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §11.2, «Attention Pooling by Similarity». https://d2l.ai/

[42] L. Zheng *et al.* Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. En *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks*, 2023. https://arxiv.org/abs/2306.05685